EduAirControl — Plan de Pruebas

Servicio Nacional de Aprendizaje – SENA Industria
Programa: Tecnólogo en Análisis y Desarrollo de Software
Ficha: 3145556
Proyecto: EduAirControl
Integrantes: Jhon Jairo Cortez Vargas; María de los Ángeles Olaya García
Instructora: Karol Daniela Correa
Ciudad: Neiva, Huila
Versión del plan: 1.0
Fecha: 9 de octubre de 2026

---

1. Propósito

Este plan define las pruebas necesarias para verificar que EduAirControl cumple sus funciones de gestión, monitoreo y control de ambientes educativos mediante IoT. Se validan el frontend web, la aplicación móvil, el API Gateway, los microservicios, la persistencia en PostgreSQL, Redis, RabbitMQ, la seguridad, los servicios de integración y el prototipo con ESP32.

El plan también establece los criterios para documentar resultados y organizar las capturas, videos, logs y reportes en la carpeta `07_Pruebas/Evidencias/`.

2. Alcance

2.1 Incluido

- Registro, inicio de sesión, renovación y cierre de sesión.
- Roles `SUPER_ADMIN`, `ADMIN`, `USER` y `VIEWER`.
- Gestión de usuarios, instituciones, ambientes y sensores.
- Consulta del dashboard, mediciones, análisis, umbrales, alertas y notificaciones.
- Favoritos, perfil, preferencias, idiomas, tema y accesibilidad.
- API Gateway, rutas públicas, rutas protegidas, JWT y rate limiting.
- Microservicios de seguridad, aulas, sensores, monitoreo, usuarios y experiencia.
- PostgreSQL, Liquibase, Redis y RabbitMQ.
- Frontend React/Vite y aplicación móvil Expo/React Native.
- Comunicación BLE de aprovisionamiento del ESP32.

- Lecturas de prueba del DHT11 y KY-038.
- Construcción Docker, smoke test, pruebas Maven y pipeline CI.

2.2 Fuera del alcance o pendiente de validación

- Certificación metrológica de los sensores.
- Uso del MQ-135 como medidor calibrado de CO_2 en ppm.
- Firmware productivo definitivo que envía mediciones al API.
- Pruebas de carga productivas y pruebas de penetración certificadas.
- Recuperación ante desastre en infraestructura real.
- Validación de proveedores sociales Google/Facebook con credenciales de producción.

> **Importante:** una prueba documentada como “pendiente” no debe registrarse como aprobada. Las evidencias deben identificar ambiente, fecha, versión, responsable y resultado real.

3. Referencias de la prueba

- Manual técnico de EduAirControl.
- Manual de instalación de EduAirControl.
- Documento `08_Calidad_Mejoras`.
- Documento `09_Implementacion`.
- Contratos OpenAPI en `Docs-ea-control/07-api/contracts/openapi/`.
- Pruebas automatizadas en `microservicios/*/src/test/`.
- Compose en `microservicios/ea-control-infra/docker-compose.yml`.
- Script `microservicios/ea-control-infra/scripts/wait-for-services.sh`.
- Script `microservicios/ea-control-infra/scripts/smoke.sh`.

4. Elementos que se probarán

Código	Elemento	Tecnología o ubicación
WEB	Aplicación web	React 19, Vite 8, Nginx
MOB	Aplicación móvil	Expo 54, React Native 0.81
GW	API Gateway	Spring Cloud Gateway, puerto 8080
SEC	Seguridad e identidad	`ms-security`, puerto 8081
CLA	Ambientes y aulas	`ms-classroom-management`, puerto 3002
MON	Monitoreo	`ms-environment-monitoring`, puerto 3003
SEN	Sensores	`ms-sensor-management`, puerto 3004
USR	Perfiles	`ms-user-management`, puerto 3007
UX	Preferencias y favoritos	`ms-user-experience`, puerto 3006
DB	Persistencia	PostgreSQL 15 + Liquibase
MSG	Mensajería y caché	RabbitMQ 3.13 + Redis 7
IOT	Prototipo físico	ESP32, DHT11, KY-038 y MQ-135
OPS	Implementación	Docker Compose, CI y Kubernetes

5. Ambientes de prueba

Ambiente	Uso	Datos	Criterio
Local	Pruebas unitarias, integración y desarrollo	Sintéticos	No usar datos personales reales
Dev	Integración continua	Sintéticos	Se actualiza desde ramas de desarrollo
Staging	Aceptación y demostración	Anonimizados o sintéticos	Debe asemejarse a la configuración final
Físico IoT	Pruebas de sensores	Lecturas controladas	Prototipo supervisado y con alimentación segura

6. Requisitos de entrada

Antes de iniciar la ejecución:

- [ ] El código de la versión a probar está identificado por commit o etiqueta.
- [ ] Docker Desktop funciona y tiene suficiente memoria.
- [ ] Los puertos requeridos están libres.
- [ ] Las variables `.env` contienen valores locales, sin secretos reales.
- [ ] Las bases de datos de prueba están disponibles.
- [ ] Los changelogs Liquibase están aplicados.
- [ ] El frontend puede comunicarse con el gateway.
- [ ] Para la app móvil, el teléfono o emulador tiene permisos de red.
- [ ] Para IoT, se verificaron voltajes, tierra común y continuidad.
- [ ] Se creó una carpeta de evidencias para la ejecución actual.

7. Estrategia de pruebas

7.1 Niveles

1. **Pruebas unitarias:** validan clases, servicios, reglas de negocio y modelos.

2. **Pruebas de integración:** validan controladores, persistencia, JWT, Redis y mensajería.
3. **Pruebas de contrato:** comparan las respuestas con los contratos OpenAPI.
4. **Pruebas funcionales:** validan los flujos desde web y móvil.
5. **Pruebas de sistema:** validan la operación completa desde cliente hasta base de datos.
6. **Pruebas IoT:** verifican lectura física, conectividad y trazabilidad de medición.
7. **Pruebas de regresión:** repiten los flujos esenciales después de cada cambio.
8. **Pruebas no funcionales:** revisan seguridad básica, disponibilidad, rendimiento y recuperación.

7.2 Técnicas

- Partición de equivalencia.
- Valores límite.
- Pruebas negativas.
- Pruebas basadas en roles.
- Pruebas de caja negra sobre API y pantallas.
- Pruebas automatizadas con JUnit y Spring Boot Test.
- Pruebas de humo después del despliegue.
- Comparación de resultados esperados y observados.

8. Pruebas automatizadas existentes

Los servicios ya incluyen pruebas en sus carpetas `src/test`. Deben ejecutarse con:

```
./mvnw -B verify
```

En Windows:

```
mvnw.cmd -B verify
```

Ejemplos de cobertura funcional existente:

Servicio	Pruebas identificadas
API Gateway	Gateway, rutas públicas, JWT, JWKS, correlation ID y rate limit
Security	Autenticación, contraseñas, OAuth2, JWT, instituciones, Super Admin y blacklist Redis
Classroom	Campus, ambientes educativos, controladores y eventos
Sensor	Catálogo, sensores, instalaciones, variables, controlador y eventos
Monitoring	Dashboard, análisis, alertas, umbrales, mediciones, backfill y consumidor RabbitMQ
User Management	Servicio y controlador de perfil
User Experience	Controlador, backfill y contexto PostgreSQL

El resultado de cada `verify` debe conservarse en la evidencia de la ejecución. Si una prueba es deshabilitada, debe registrarse la razón y la fecha de corrección prevista.

9. Matriz de casos funcionales

9.1 Identidad y seguridad

ID	Caso de prueba	Precondición	Pasos resumidos	Resultado esperado	Prioridad
SEC-001	Registrar usuario válido	API disponible	Enviar correo, nombre y contraseña válidos	Usuario creado y respuesta exitosa	Alta
SEC-002	Rechazar correo duplicado	Usuario existente	Registrar el mismo correo	Error controlado sin duplicar registro	Alta
SEC-003	Iniciar sesión válido	Usuario activo	Enviar credenciales correctas	Se entrega access token y refresh token	Alta
SEC-004	Rechazar credenciales inválidas	Usuario existente	Enviar contraseña incorrecta	Respuesta 401 sin filtrar información sensible	Alta
SEC-005	Renovar sesión	Refresh token válido	Solicitar renovación	Se entrega token válido o respuesta definida	Alta
SEC-006	Cerrar sesión	Sesión activa	Ejecutar logout	Token revocado; reutilización rechazada	Alta
SEC-007	Restablecer contraseña	Correo de prueba	Solicitar y completar	Token temporal	Media

ID	Caso de prueba	Precondición	Pasos resumidos	Resultado esperado	Prioridad
		disponible	recuperación	controlado y contraseña actualizada	
SEC-008	Validar permisos ADMIN	Usuario ADMIN	Acceder a gestión	Acceso permitido	Alta
SEC-009	Bloquear gestión a USER	Usuario USER	Intentar crear ambiente o sensor	Respuesta 403 o redirección controlada	Alta
SEC-010	Administrar roles como SUPER_ADMIN	SUPER_ADMIN activo	Asignar ADMIN a usuario de prueba	Rol actualizado y auditado	Alta
SEC-011	Probar rate limiting	Gateway activo	Superar solicitudes permitidas	Solicitudes limitadas sin caída del servicio	Media
SEC-012	No exponer secretos	Logs disponibles	Revisar logs y respuestas	No aparecen contraseñas, tokens completos ni claves	Alta

9.2 Ambientes, sensores y monitoreo

ID	Caso de prueba	Resultado esperado	Prioridad
ENV-001	Crear campus o institución	Registro creado con validaciones	Alta
ENV-002	Crear ambiente educativo	Ambiente asociado a institución y tipo	Alta
ENV-003	Editar ambiente	Cambios persistidos y visibles	Alta
ENV-004	Buscar y filtrar ambientes	Resultados coinciden con los criterios	Media
SEN-001	Registrar sensor	Sensor creado con variable y estado	Alta
SEN-002	Asociar sensor a ambiente	Instalación registrada correctamente	Alta
SEN-003	Editar o desactivar sensor	Estado actualizado y no se aceptan lecturas indebidas	Alta
SEN-004	Registrar dispositivo IoT	Se genera una clave que no se vuelve a mostrar completa	Alta
MON-001	Ingresar medición válida	Medición aceptada y persistida	Alta
MON-002	Rechazar medición sin API key	Respuesta $^{401}/_{403}$ y no se persiste	Alta
MON-003	Consultar resumen del dashboard	Indicadores y estados correctos	Alta
MON-004	Consultar serie histórica	Serie ordenada por fecha y variable	Alta
MON-005	Activar umbral de alerta	Se crea alerta según regla configurada	Alta

ID	Caso de prueba	Resultado esperado	Prioridad
MON-006	Consultar notificaciones	Alertas, advertencias y resumen visibles	Media
MON-007	Procesar evento RabbitMQ	Evento consumido sin duplicar información	Alta

9.3 Frontend web y móvil

ID	Caso de prueba	Resultado esperado	Prioridad
WEB-001	Abrir landing	La página carga sin errores de consola bloqueantes	Alta
WEB-002	Login desde web	Usuario llega al dashboard según su rol	Alta
WEB-003	Navegar dashboard	Gráficos, indicadores y filtros cargan	Alta
WEB-004	Consultar ambientes	Lista, búsqueda y detalle funcionan	Alta
WEB-005	Administrar desde panel	ADMIN puede crear, editar y filtrar	Alta
WEB-006	Guardar favorito	Favorito aparece solo para el usuario actual	Media
WEB-007	Cambiar preferencias	Tema, idioma y accesibilidad persisten	Media
MOB-001	Iniciar Expo	Metro inicia sin error de dependencias	Alta
MOB-002	Login móvil	La sesión se almacena en AsyncStorage	Alta
MOB-003	Acceder a API desde teléfono	La URL LAN responde correctamente	Alta
MOB-004	Navegar dashboard móvil	Datos consultados y representados	Alta
MOB-005	Aprovisionar ESP32 por BLE	Se detecta `ESP32_Config` y se envían datos definidos	Media
MOB-006	Revocar permisos BLE	La aplicación muestra error controlado y no se bloquea	Media

9.4 IoT y hardware

ID	Caso de prueba	Resultado esperado	Prioridad
IOT-001	Leer DHT11	Temperatura y humedad aparecen en serial a 115200	Alta
IOT-002	Desconectar DHT11	Se muestra error controlado, sin valores inventados	Media
IOT-003	Leer KY-038 en silencio	Valor estable dentro del comportamiento esperado	Media
IOT-004	Generar sonido cerca del KY-038	La salida AO cambia de forma observable	Media
IOT-005	Validar MQ-135	Se verifica variación relativa sin afirmar ppm calibradas	Media
IOT-006	Probar voltaje ADC MQ-135	No se supera el máximo permitido del ESP32	Alta
IOT-007	Reconectar Wi-Fi	Firmware recupera conectividad o registra el fallo	Alta
IOT-008	Enviar medición a API	API valida la clave y persiste la lectura	Alta; pendiente de firmware

10. Pruebas de API

La API principal está disponible localmente en `http://localhost:8080`. Cada prueba debe registrar método, ruta, cuerpo anonimizado, código HTTP, respuesta y tiempo de respuesta.

Ejemplos de verificaciones:

```
curl -i http://localhost:8080/health
curl -i http://localhost:8081/api/v1/auth/health
curl -i http://localhost:3002/health
curl -i http://localhost:3004/health
```

Para pruebas autenticadas:

1. Crear un usuario de prueba.
2. Iniciar sesión y guardar el token solo en la máquina de pruebas.
3. Ejecutar la solicitud con `Authorization: Bearer <token>`.
4. Eliminar o invalidar el token al finalizar.
5. No incluir tokens reales en capturas, documentos o repositorios.

Las rutas críticas incluyen:

- `/auth/login`.
- `/auth/register`.
- `/api/v1/profile`.
- `/api/v1/environments`.
- `/api/v1/sensors`.
- `/api/v1/devices`.
- `/ingest/v1/measurements`.
- `/api/v1/dashboard/summary`.
- `/api/v1/dashboard/series`.
- `/api/v1/notifications`.
- `/api/v1/favorites`.
- `/health`.

11. Pruebas no funcionales

11.1 Rendimiento

- Medir tiempo de respuesta de login, dashboard y consulta histórica.
- Medir p95 de endpoints principales en staging.
- Verificar que el gateway limite solicitudes sin consumir toda la memoria.
- Probar consulta con distintos rangos de fechas.
- Registrar número de usuarios concurrentes, duración y errores.

Como referencia inicial de aceptación para una demostración local:

Indicador	Criterio sugerido
Health check	Respuesta exitosa en menos de 1 segundo en local
Login	Menos de 2 segundos en condiciones normales
Dashboard	Menos de 3 segundos con datos de prueba
Error HTTP 5xx	0 durante el flujo crítico
Disponibilidad durante prueba	100% del tiempo planificado

Estos valores son criterios de aceptación del proyecto, no un SLA productivo.

11.2 Seguridad

- Verificar autorización por rol.
- Probar expiración, firma y revocación JWT.
- Probar que una API key inválida no ingrese mediciones.
- Revisar CORS con orígenes permitidos y no permitidos.
- Revisar que no se registren secretos.
- Revisar dependencias y vulnerabilidades de imágenes.
- Confirmar que las credenciales de prueba no se reutilicen en producción.

11.3 Disponibilidad y recuperación

- Reiniciar un servicio y verificar que vuelva a estar saludable.
- Detener Redis y observar el comportamiento del gateway.
- Detener RabbitMQ y verificar reintentos o mensajes pendientes.
- Reiniciar PostgreSQL sin eliminar volúmenes.
- Restaurar una copia de prueba en una base aislada.
- Ejecutar rollback de una imagen en Kubernetes en staging.

12. Criterios de entrada y salida

12.1 Entrada

La ejecución puede comenzar cuando el ambiente, la versión, las dependencias, las variables y los datos de prueba estén definidos y el stack mínimo responda.

12.2 Salida

Una ronda de pruebas se puede cerrar cuando:

- todos los casos de prioridad alta fueron ejecutados;
- no existen defectos críticos o bloqueantes abiertos;
- los defectos medios tienen responsable y fecha;
- se conservaron evidencias suficientes;
- el smoke test terminó exitosamente;
- las pruebas automatizadas fueron ejecutadas;
- las pruebas pendientes quedaron explícitamente marcadas;
- el responsable técnico y el responsable funcional revisaron el reporte.

12.3 Clasificación de resultados

Resultado	Significado
PASS	Resultado esperado obtenido
FAIL	Resultado diferente al esperado
BLOCKED	No se puede ejecutar por dependencia o ambiente
NOT RUN	Programada pero no ejecutada
NOT APPLICABLE	No aplica a esta versión, con justificación

13. Gestión de defectos

Cada defecto debe registrarse con:

- identificador;

- fecha y ambiente;
- versión o commit;
- caso de prueba relacionado;
- pasos para reproducir;
- resultado esperado y actual;
- evidencia;
- severidad y prioridad;
- responsable;
- estado;
- fecha de corrección y de retest.

Severidad	Descripción
Crítica	Impide iniciar el sistema, compromete datos o permite acceso no autorizado
Alta	Bloquea una función principal sin alternativa razonable
Media	Afecta una función secundaria o existe una alternativa
Baja	Impacto visual, textual o de conveniencia

14. Formato de reporte de ejecución

Cada ejecución debe crear un archivo con el formato:

```
Ejecucion_YYYY-MM-DD_<ambiente>_<version>.md
```

Debe incluir:

```
# Reporte de ejecución
- Fecha:
- Responsable:
- Ambiente:
- Commit/tag:
- Sistema operativo:
- Versión Docker/Java/Node:
- Casos ejecutados:
- PASS:
- FAIL:
- BLOCKED:
- NOT RUN:
- Defectos abiertos:
- Conclusión:
```

15. Organización de evidencias

La carpeta debe conservar una estructura por ejecución:

```
07_Pruebas/
├── Plan_Pruebas.md
├── Plan_Pruebas.pdf
└── Evidencias/
    ├── README.md
    ├── 2026-10-09_local_v1.0/
    │   ├── 00_Reporte_Ejecucion.md
    │   ├── 01_Ambiente/
    │   ├── 02_Autenticacion/
    │   ├── 03_Ambientes_Sensores/
    │   ├── 04_Monitoreo_Alertas/
    │   ├── 05_Web/
    │   ├── 06_Movil/
    │   ├── 07_IoT/
    │   ├── 08_API_Logs/
    │   └── 09_CI_Despliegue/
    └── Plantilla_Evidencia.md
```

Nomenclatura recomendada:

```
<caso>_<resultado>_<fecha>_<consecutivo>.<extension>
```

Ejemplos:

- `SEC-003_PASS_2026-10-09_01.png`.
- `MON-003_PASS_2026-10-09_01.mp4`.
- `IOT-001_PASS_2026-10-09_01.jpg`.
- `GW-HEALTH_PASS_2026-10-09_01.txt`.
- `WEB-002_FAIL_2026-10-09_01.md`.

Las capturas deben ocultar contraseñas, tokens, API keys, correos personales y datos sensibles. Para logs, usar archivos `.txt` o `.log` con la información sensible anonimizada.

16. Aprobación

Rol	Nombre	Firma o aprobación	Fecha
Responsable de pruebas	Por asignar	Pendiente	Pendiente
Responsable técnico	Por asignar	Pendiente	Pendiente
Responsable funcional	Por asignar	Pendiente	Pendiente
Instructor/a	Karol Daniela Correa	Pendiente	Pendiente

17. Conclusión

Este plan permite evaluar EduAirControl de forma progresiva: primero se comprueba la unidad de cada servicio, luego la integración de API, después los flujos web y móviles, y finalmente la cadena IoT y el despliegue. El sistema solo debe declararse listo para una entrega cuando los casos críticos tengan resultado satisfactorio y las limitaciones pendientes estén aceptadas y documentadas.